Edge cases (an ambiguous routing decision) — Solution

> Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `supervisor_routing_practice.py`: routing looks fine right up until two specialists both plausibly fit the same task. This file constructs exactly that ambiguous case on purpose, runs it repeatedly to count how often the supervisor's choice actually wobbles between the two, and adds a way to recover when it picks wrong. **If not:** Project 4's Research-vs-Analysis boundary would misroute in production, with no test that ever caught it happening.

Solution

Story: a single run on one task can't tell a genuinely stable routing choice from a lucky coin flip; running the exact same ambiguous task three times and comparing the choices can. **If not:** an inconsistent router would pass every one-shot test cleanly and then fail at random for real users.

```
# supervisor_routing_practice.py
from typing import TypedDict

from langchain_openai import ChatOpenAI
from langgraph.graph import StateGraph, START, END


class RouteState(TypedDict):
    task: str
    picked: str


def supervisor(state: RouteState) -> dict:
    # temperature > 0 on purpose, so an ambiguous task can wobble between
    # runs — which is exactly what this exercise measures.
    model = ChatOpenAI(model="gpt-4o-mini", temperature=1.0)
    prompt = (
        "Route this task to exactly one specialist. Reply with only "
        "'research_agent' (finding out facts or causes) or "
        "'analysis_agent' (interpreting data we already have).\n\n"
        f"Task: {state['task']}"
    )
```

```
    choice = model.invoke(prompt).content.strip()
    if "analysis" in choice:
        return {"picked": "analysis_agent"}
    return {"picked": "research_agent"}


def build_graph():
    builder = StateGraph(RouteState)
    builder.add_node("supervisor", supervisor)
    builder.add_edge(START, "supervisor")
    builder.add_edge("supervisor", END)
    return builder.compile()


graph = build_graph()


def run_ambiguity_check(prompts: list[str], attempts: int = 3) -> dict:
    tally = {}
    for prompt in prompts:
        picks = []
        for _ in range(attempts):
            result = graph.invoke({"task": prompt, "picked": ""})
            picks.append(result["picked"])
        tally[prompt] = picks
    return tally

# Run each ambiguous task a few times and see if the routing is stable.
test_prompts = [
    "find out why sales dropped last quarter",
    "what does this survey data actually tell us",
    "look into the causes of the outage",
    "explain what happened to our conversion rate",
    "dig into the reasons behind the delay",
]
tally = run_ambiguity_check(test_prompts)
for prompt, picks in tally.items():
    consistent = len(set(picks)) == 1
    label = "consistent"
    if not consistent:
        label = "INCONSISTENT"
    print(f"{prompt!r}:")
    print(f"  {picks} ({label})")
```

Expected output (exact picks vary by run):

```
'find out why sales dropped last quarter':
  ['research_agent', 'research_agent', 'analysis_agent'] (INCONSISTENT)
'what does this survey data actually tell us':
  ['analysis_agent', 'analysis_agent', 'analysis_agent'] (consistent)
'look into the causes of the outage':
  ['research_agent', 'research_agent', 'research_agent'] (consistent)
'explain what happened to our conversion rate':
  ['analysis_agent', 'research_agent', 'analysis_agent'] (INCONSISTENT)
'dig into the reasons behind the delay':
  ['research_agent', 'research_agent', 'research_agent'] (consistent)
```